

**Project Report
on
QUIZ MANAGEMENT SYSTEM**



Submitted in partial fulfillment for the award of
Core Java Internship

from **Sunbeam Institute of Information Technology
(Pune)**

Guided by:
Name - Ms. Vishakha Reddy

Presented By:

Name - Deepesh Jangid

Roll No. - 97649

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our Project Guide, Mr./Ms. **Ms. Vishakha Reddy**, for providing valuable guidance, continuous support, and encouragement throughout the development of this project. Their expertise, constructive suggestions, and motivation helped us successfully complete our project.

We are also thankful to the Course Coordinator and all the faculty members of Sunbeam Institute of Information Technology, Pune, for their constant support and for providing us with the necessary resources and learning environment to carry out this work.

We extend our heartfelt thanks to our classmates and friends for their cooperation, suggestions, and assistance during the project. Their support and teamwork greatly contributed to the successful completion of this project.

Finally, we would like to express our sincere gratitude to our parents and family members for their unconditional encouragement, patience, and moral support throughout our academic journey.

This project has been a valuable learning experience, helping us enhance our technical knowledge, practical skills, and problem-solving abilities.

Team Members

TABLE OF CONTENTS

1. Introduction	3
2. Objectives	4
3. System Requirements	5
4. System Architecture	6
5. Database Design	7
6. Module Description	9
7. Class Design & UML	11
8. Technology Stack	13
9. Exception Handling	14
10. User Interface Description	15
11. Data Flow	16
12. Testing	17
13. Limitations & Future Enhancements	18
14. Conclusion	19
15. References	20

1. INTRODUCTION

1.1 Overview

The Quiz Management System (QMS) is a desktop-based Java application designed to automate the process of creating, administering, and evaluating quizzes for educational institutions. Built using Java Swing for the graphical user interface (GUI) and MySQL as the backend database, the system enables administrators to manage quizzes and students to attempt them online with instant score feedback.

Traditional quiz and examination processes are labour-intensive and error-prone. This system eliminates paper-based quizzes by providing a fully digital platform where quizzes can be created by uploading structured CSV or TXT files, and students can attempt them through an intuitive GUI.

1.2 Problem Statement

Educational institutions face several challenges with manual quiz conduction:

- Time-consuming paper-based quiz creation and evaluation
- Lack of instant feedback for students on their performance
- Difficulty in storing and retrieving historical quiz records
- No centralised system for managing multiple quizzes and student attempts
- Inconsistency in result computation and data entry errors

The Quiz Management System addresses all the above challenges by providing a single, integrated desktop application.

1.3 Scope

The scope of this project covers:

- User authentication for both Admin and Student roles
- Quiz creation from structured file input (CSV/TXT)
- Student registration, login, and quiz attempt functionality
- Score computation and storage in the MySQL database
- Admin control for managing (create, list, delete) quizzes and viewing all results

2. OBJECTIVES

The primary objectives of the Quiz Management System are:

2.1 Primary Objectives

- To develop a fully functional desktop quiz application using Java Swing
- To enable administrators to create quizzes by uploading CSV or TXT question files
- To provide students with a user-friendly interface to attempt quizzes and view scores
- To integrate MySQL as a persistent backend for storing users, quizzes, questions, and attempts
- To implement role-based access control (RBAC) separating Admin and Student functionalities

2.2 Secondary Objectives

- To implement custom exception handling for robust error management
- To demonstrate a clean DAO (Data Access Object) pattern for database interaction
- To ensure quiz integrity by validating file formats before quiz creation
- To allow admins to delete quizzes to manage quiz repository
- To allow students to view their past attempt scores

2.3 Expected Outcomes

Upon successful completion, the system is expected to:

- Reduce the time needed to set up and evaluate a quiz from hours to minutes
- Provide instant score feedback to students immediately after quiz submission
- Maintain a secure, centralised database of all quizzes, users, and attempts
- Demonstrate sound software engineering principles including OOPS, DAO pattern, and MVC architecture

3. SYSTEM REQUIREMENTS

3.1 Software Requirements

Software	Version / Details
Operating System	Windows 10 / Linux Ubuntu 20.04+
Java Development Kit (JDK)	JDK 11 or above
Java Runtime Environment (JRE)	JRE 11 or above
Database	MySQL 8.0+
JDBC Driver	mysql-connector-java-8.x.jar
Build Tool	Apache Maven 3.6+
IDE (Development)	IntelliJ IDEA / Eclipse / NetBeans
Database Client (Optional)	MySQL Workbench 8.0

3.2 Network Requirements

The application runs entirely on a local machine or a local area network (LAN). No internet connection is required. MySQL must be accessible on localhost:3306 with the credentials configured in DbUtil.java.

4. SYSTEM ARCHITECTURE

4.1 Architectural Pattern

The Quiz Management System follows a three-tier architecture that cleanly separates the presentation, business logic, and data access layers:

Layer	Description	Technologies Used
Presentation Layer	Graphical User Interface built using Java Swing	JFrame, JButton, JLabel, JTable, JPanel
Business Logic Layer	DAO Interfaces and DAO Implementations	Java Classes, OOPS, Exception Handling
Data Access Layer	MySQL database connected via JDBC	JDBC, PreparedStatement, ResultSet

4.2 Package Structure

The project follows a well-organised Maven package structure:

Package	Purpose
quizmanagementsystem.main	All JFrame GUI classes (HomeFrame, AdminDashboard, etc.)
quizmanagementsystem.model	POJO/Entity classes (User, Quiz, Question, Attempt, Role)
quizmanagementsystem.dao	Interface declarations for data access operations
quizmanagementsystem.daoimpl	Concrete JDBC implementations of DAO interfaces
quizmanagementsystem.util	Utility classes (DbUtil for DB connection)
quizmanagementsystem.Exception	Custom Exception classes

4.3 Application Flow

The high-level application flow is as follows:

- Application starts at Main.java, which launches HomeFrame
- HomeFrame provides options: Admin Login, Student Register, Student Login, and Exit
- Admin logs in via AdminLoginFrame → redirected to AdminDashboard
- Student registers via StudentRegisterFrame or logs in → redirected to StudentDashboard
- Admin can Create Quiz, List Quizzes, Delete Quiz, View Results, or Logout
- Student can View Quizzes, Take (Attempt) a Quiz, View Scores, or Logout

5. DATABASE DESIGN

5.1 Database Overview

The database is named quiz_management and is hosted on MySQL. It consists of five tables that store all application data persistently.

5.2 Entity-Relationship Summary

- A User can have the role of ADMIN or STUDENT
- A Quiz contains one or more Questions
- A Student (User with STUDENT role) can Attempt one or more Quizzes
- Each Attempt records the student_id, quiz_id, and score
- The Results table mirrors the Attempt table and is reserved for future reporting

5.3 Table Definitions

5.3.1 users

Column	Data Type	Constraint	Description
user_id	INT	PK, AUTO_INCREMENT	Unique identifier for each user
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(100)	NOT NULL, UNIQUE	Email address used for login
password	VARCHAR(200)	NOT NULL	User password (plain text)
role	ENUM('ADMIN','STUDENT')	NOT NULL	Determines access level

5.3.2 quizzes

Column	Data Type	Constraint	Description
quiz_id	INT	PK, AUTO_INCREMENT	Unique identifier for each quiz
title	VARCHAR(200)	NOT NULL	Title or topic of the quiz

5.3.3 questions

Column	Data Type	Constraint	Description
question_id	INT	PK, AUTO_INCREMENT	Unique question ID
quize_id	INT	NOT NULL	Foreign key to quizzes table
question_text	TEXT	NOT NULL	The question statement
option_a	VARCHAR(200)	NOT NULL	First option
option_b	VARCHAR(200)	NOT NULL	Second option
option_c	VARCHAR(200)	NOT NULL	Third option
option_d	VARCHAR(200)	NOT NULL	Fourth option
correct_answer	VARCHAR(200)	NOT NULL	Correct answer key (A/B/C/D)

5.3.4 attempt

Column	Data Type	Constraint	Description
attempt_id	INT	PK, AUTO_INCREMENT	Unique attempt record ID
student_id	INT	NOT NULL	ID of the student attempting
quiz_id	INT	NOT NULL	ID of the quiz being attempted
score	INT	NOT NULL	Score obtained in the attempt

5.3.5 results

The results table has the same schema as attempt (result_id, student_id, quiz_id, score). It is currently empty and reserved for future extended reporting features, such as storing detailed question-by-question responses.

5.4 Sample Data Statistics

Table	Record Count	Notes
users	42	Includes 8 ADMIN and 34 STUDENT records
quizzes	15	Quiz IDs 7–22 (some IDs skipped due to deletions)
questions	60+	10 questions per quiz (Java topic)
attempt	8	Quiz attempts recorded by students
results	0	Reserved for future use

6. MODULE DESCRIPTION

6.1 Admin Module

The Admin Module is accessible only to users with the ADMIN role. It provides the following sub-modules:

6.1.1 Admin Login

Handled by AdminLoginFrame.java. The admin enters their email and password. These credentials are verified against the users table in MySQL. Upon successful authentication, the AdminDashboard is loaded. On failure, an error message is displayed.

6.1.2 Create Quiz

Handled by CreateQuiz.java. The admin provides a quiz title and a file path pointing to a CSV or TXT file containing questions. The system validates the inputs — it checks that neither field is empty and that the file has a .csv or .txt extension. The file is read line by line, with each group of 6 lines forming one question (question text, 4 options, correct answer). Questions are inserted into the questions table with the new quiz's auto-generated quiz_id.

6.1.3 List Quizzes

Handled by ListQuizFram.java. Retrieves all records from the quizzes table and displays them to the admin.

6.1.4 Delete Quiz

Handled by DeleteQuiz.java. Allows the admin to delete a quiz by entering a quiz ID. Executes a DELETE SQL statement and throws a QuizNotFoundException if the quiz does not exist.

6.1.5 View Results

Handled by ViewResults.java. Retrieves all records from the attempt table and displays the full list of attempts, showing student IDs, quiz IDs, and scores.

6.2 Student Module

The Student Module is accessible to users with the STUDENT role.

6.2.1 Student Registration

Handled by StudentRegisterFrame.java. New students can register by providing their name, email, and password. The system inserts a new record into the users table with role = 'STUDENT'.

6.2.2 Student Login

Handled by AdminLoginFrame.java (shared for both roles). The system queries the users table by email and password. If authenticated, the student is directed to StudentDashboard, receiving their user_id as a session identifier passed between frames.

6.2.3 View Quizzes

Handled by ViewQuizFrame.java. Retrieves all available quizzes from the quizzes table and displays them for the student's reference.

6.2.4 Attempt Quiz

Handled by AttemptQuizFrame.java. The student selects a quiz by ID. The system fetches all questions for that quiz_id from the questions table. Questions are presented one by one with multiple choice options. Upon submission, the answers are evaluated, the score is computed, and an attempt record is inserted into the attempt table.

6.2.5 View Score

Handled by ViewScore.java. Queries the attempt table filtered by the student's student_id and displays all past quiz attempts and scores.

7. CLASS DESIGN & UML DESCRIPTION

7.1 Model Classes

Class	Attributes	Purpose
User	id, name, email, password, role, attemptMap	Represents a system user (Admin or Student)
Quiz	id, title	Represents a quiz entity
Question	questionId, quizId, questionText, optionA-D, correctAnswer	Represents a single quiz question
Attempt	studentId, quizId, score	Records a student's quiz attempt and score
Role (Enum)	ADMIN, STUDENT	Enumeration of user roles

7.2 DAO Interface – QuizDao

The QuizDao interface defines the contract for all quiz-related data operations:

- `create_quiz(String title, String path)` – Creates a new quiz from a file
- `listQuiz()` – Returns a `Set<Quiz>` of all quizzes
- `viewResults()` – Fetches all quiz attempt results
- `attemptQuiz(int quizId)` – Records a student's attempt
- `viewScore(int student_id)` – Fetches scores for a specific student

7.3 DAO Implementation – QuizDaoimpl

QuizDaoimpl provides the concrete JDBC implementation for each method. It holds a Connection object obtained from `DbUtil.getConnection()` and uses PreparedStatement for all database queries to prevent SQL injection.

7.4 DAO Implementation – userDaoimpl

userDaoimpl handles all user-related operations including login authentication (querying users by email and password) and student registration (inserting new users).

7.5 Utility Class – DbUtil

DbUtil is a static utility class that encapsulates the JDBC database connection logic. It loads the MySQL JDBC driver via a static initialiser block and provides a `getConnection()` factory method:

- DB Driver: com.mysql.cj.jdbc.Driver
- DB URL: jdbc:mysql://localhost:3306/quize_management
- DB User: root | DB Password: 1234

7.6 GUI Classes Summary

Class	Type	Description
Main.java	Entry Point	Launches the application by instantiating HomeFrame
HomeFrame.java	JFrame	Main landing page with role-based navigation buttons
AdminLoginFrame.java	JFrame	Shared login form for Admin and Student authentication
AdminDashboard.java	JFrame	Admin's menu with Create, List, Delete Quiz, View Results, Logout
StudentRegisterFrame.java	JFrame	Student registration form
StudentDashboard.java	JFrame	Student menu with View Quiz, Take Quiz, View Score, Logout
CreateQuiz.java	JFrame	Form to enter quiz title and file path for quiz creation
ListQuizFram.java	JFrame	Displays all available quizzes in a table
DeleteQuiz.java	JFrame	Form to enter quiz ID for deletion
ViewResults.java	JFrame	Displays all student attempt records
AttemptQuizFrame.java	JFrame	Quiz-taking interface showing questions and options
ViewQuizFrame.java	JFrame	Shows list of available quizzes for the student
ViewScore.java	JFrame	Shows the student's past scores
QuestionFrame.java	JFrame	Renders individual question with MCQ options during a quiz

8. TECHNOLOGY STACK

8.1 Programming Language – Java

Java (JDK 11+) is used as the primary programming language. Java is platform-independent, object-oriented, and provides robust libraries for GUI development (Swing) and database connectivity (JDBC). The project leverages core Java OOP concepts including inheritance, encapsulation, polymorphism, and interfaces.

8.2 Graphical User Interface – Java Swing

Java Swing is used to build the entire GUI. Swing provides a rich set of platform-independent GUI components.

Swing Component	Usage in Project
JFrame	Base window container for all screens
JButton	Navigation and action buttons (Login, Register, Create Quiz, etc.)
JLabel	Display text, titles, and icons
TextField / JPasswordField	User input for credentials and quiz details
JTable	Display tabular data (quiz lists, results, scores)
JOptionPane	Pop-up messages for success/error feedback
JPanel	Grouping and layout management of components

8.3 Database – MySQL

MySQL 8.0 is used as the Relational Database Management System (RDBMS). The database stores all persistent application data including users, quizzes, questions, and attempts. MySQL's reliability, ACID compliance, and compatibility with JDBC make it ideal for this application.

8.4 Database Connectivity – JDBC

Java Database Connectivity (JDBC) provides the API for connecting Java applications to relational databases. The application uses:

- DriverManager – To establish database connections
- Connection – Represents an active database session
- PreparedStatement – For parameterised SQL queries (prevents SQL injection)
- ResultSet – To iterate over query results

- Statement.RETURN_GENERATED_KEYS – To retrieve auto-generated quiz IDs after INSERT

8.5 Build Tool – Apache Maven

Apache Maven is used for project build management. The pom.xml file manages project metadata and the MySQL JDBC connector dependency, ensuring consistent builds across different environments.

9. EXCEPTION HANDLING

The project implements a comprehensive custom exception handling strategy using user-defined exception classes that extend the built-in Exception class.

9.1 Custom Exception Classes

Exception Class	Package	When Thrown
QuizNotFoundException	quizmanagementsystem.Exception	When a quiz with the given ID does not exist in the database (e.g., during delete)
QuizAlreadyAttemptedException	quizmanagementsystem.Exception	When a student tries to attempt a quiz they have already completed
NoQuestionsFoundException	quizmanagementsystem.Exception	When a CSV/TXT file is read but no questions are found
AuthenticationFailedException	quizmanagementsystem.Exception	When login credentials are invalid or authentication fails
InvalidInputException	quizmanagementsystem.Exception	When form inputs are empty, null, or in an invalid format

9.2 Input Validation in create_quiz()

The create_quiz() method in QuizDaoimpl performs the following validation checks before executing any database operation:

- Checks if the quiz title is empty or blank — throws a generic Exception with a descriptive message
- Checks if the file path is empty or blank — throws a generic Exception
- Validates the file extension — only .csv or .txt files are accepted; otherwise throws an Exception

9.3 Exception Propagation Strategy

All DAO methods declare throws Exception in their method signatures, propagating exceptions up to the GUI layer. The GUI classes catch these exceptions and display appropriate error messages to the user via JOptionPane.showMessageDialog(), ensuring the application never crashes silently.

10. USER INTERFACE DESCRIPTION

10.1 Design Principles

The application follows a consistent visual design language throughout all screens:

- Primary colour: Segoe UI Blue (#2980B9) for headings and primary buttons
- Background: Light blue/grey (#F0F8FF or RGB 248,250,252) for all screens
- Button colours are role-specific: Green for Create, Blue for List/View, Purple for Results, Red for Delete/Logout
- Font: Segoe UI is used consistently at sizes 16–34pt for titles and 14–16pt for buttons
- Cursor changes to HAND_CURSOR on interactive buttons for better UX
- Images/icons are loaded from an images/ directory and scaled using Image.SCALE_SMOOTH

10.2 Screen Descriptions

Screen	Size	Key Components
HomeFrame	700 × 720	Logo icon, title label, 4 navigation buttons (Admin Login, Student Register, Student Login, Exit)
AdminLoginFrame	Fixed	Email field, password field, Login button, error labels
AdminDashboard	650 × 650	Admin icon, heading, 5 coloured action buttons
CreateQuiz	Fixed	Quiz title field, file path field, Create button
ListQuizFram	Fixed	JTable showing all quiz IDs and titles, Back button
DeleteQuiz	Fixed	Quiz ID input field, Delete button, confirmation feedback
ViewResults	Fixed	JTable with attempt_Id, student_Id, quiz_Id, score columns
StudentRegisterFrame	Fixed	Name, email, password fields, Register button
StudentDashboard	650 × 700	Student icon, heading, 4 coloured action buttons with icons
ViewQuizFrame	Fixed	Table of quizzes available for attempt
AttemptQuizFrame	Fixed	Quiz ID input, question display area, radio buttons for MCQ options
ViewScore	Fixed	Table of past attempts with quiz IDs and scores

11. DATA FLOW

11.1 Admin – Create Quiz Flow

- Admin opens CreateQuiz frame and enters quiz title and CSV/TXT file path
- System validates inputs (non-empty, valid file extension)
- SQL INSERT INTO quizzes(title) is executed; generated quiz_id is retrieved
- File is read line by line; for each group of 6 lines, a question record is inserted into questions table
- Success or failure message is shown to admin

11.2 Student – Attempt Quiz Flow

- Student opens AttemptQuizFrame and enters the quiz_id
- System queries: SELECT * FROM questions WHERE quize_id = ?
- Questions are displayed one by one in QuestionFrame
- Student selects an answer (A/B/C/D) for each question
- After the last question, answers are compared to correct_answer values
- Score is calculated; INSERT INTO attempt(student_Id, quiz_Id, score) is executed
- Score is displayed to the student

11.3 Student – View Score Flow

- Student opens ViewScore frame
- System executes: SELECT * FROM attempt WHERE student_id = ?
- All past attempts are retrieved and displayed in a JTable

11.4 Admin – View Results Flow

- Admin opens ViewResults frame
- System executes: SELECT * FROM attempt
- All student attempts across all quizzes are displayed in a JTable

11.5 Authentication Flow

- User enters email and password in the login form
- System queries: SELECT * FROM users WHERE email = ? AND password = ?
- If a matching record is found, the role (ADMIN or STUDENT) is checked
- Based on role, the user is redirected to AdminDashboard or StudentDashboard
- The user_id is passed as a parameter to the dashboard for session management

12. TESTING

12.1 Testing Approach

The system was tested manually using functional and unit-level testing. Each module was tested independently before integration testing was performed.

12.2 Test Cases

Test Case ID	Module	Test Description	Expected Result	Status
TC-01	Login	Admin login with valid credentials	Redirect to AdminDashboard	PASS
TC-02	Login	Login with invalid password	Show error message	PASS
TC-03	Register	Student registration with new email	Record inserted in users table	PASS
TC-04	Register	Registration with duplicate email	Show UNIQUE constraint error	PASS
TC-05	Create Quiz	Create quiz with valid CSV file	Quiz and questions added to DB	PASS
TC-06	Create Quiz	Create quiz with empty title	Show validation error message	PASS
TC-07	Create Quiz	Create quiz with .pdf file extension	Show format error message	PASS
TC-08	List Quiz	List all quizzes	All quizzes shown in JTable	PASS
TC-09	Delete Quiz	Delete quiz with valid quiz_id	Quiz deleted from DB	PASS
TC-10	Delete Quiz	Delete quiz with invalid quiz_id	QuizNotFoundException thrown	PASS
TC-11	Attempt Quiz	Student attempts a quiz with valid quiz_id	Questions displayed correctly	PASS
TC-12	Attempt Quiz	Submit quiz answers	Score computed and saved to attempt table	PASS
TC-13	View Score	Student views their scores	All attempt records displayed	PASS
TC-14	View Results	Admin views all results	All attempt records from DB shown	PASS

12.3 Database-Level Testing

The database was tested using MySQL CLI to verify data integrity:

- Confirmed UNIQUE constraint on users.email prevents duplicate registrations
- Verified AUTO_INCREMENT on all primary keys (user_id, quiz_id, question_id, attempt_id)
- Confirmed that cascaded deletion removes associated questions when a quiz is deleted
- Tested that the attempt table correctly records student_id, quiz_id, and computed score

13. LIMITATIONS & FUTURE ENHANCEMENTS

13.1 Current Limitations

#	Limitation	Impact
1	Passwords stored in plain text	Security risk – passwords should be hashed (e.g., BCrypt)
2	No timer for quiz attempts	Students can take unlimited time to complete a quiz
3	No quiz attempt restriction (same quiz multiple times)	Students can re-attempt the same quiz, inflating scores
4	No foreign key constraints between tables	Data integrity relies on application logic, not DB constraints
5	Questions table uses 'quize_id' (typo)	Minor inconsistency in column naming conventions
6	No email validation on registration	Invalid email formats can be entered (e.g., 'fhdsu' as email)
7	Desktop-only application	Not accessible on mobile or web browsers
8	Results table currently unused	Functionality gap between schema and implementation

13.2 Future Enhancements

- Implement password hashing using BCrypt or SHA-256 for secure storage
- Add a countdown timer for quiz attempts to simulate real exam conditions
- Enforce one-attempt-per-quiz rule using the QuizAlreadyAttemptedException already defined
- Add database-level foreign key constraints between quizzes→questions and users→attempt tables
- Develop a web-based version using Java Spring Boot + React.js for browser accessibility
- Implement a quiz leaderboard showing top scorers per quiz
- Add email-based OTP verification during student registration
- Support for image-based questions and rich text formatting
- Add a dedicated admin panel for managing user accounts (activate/deactivate)
- Implement detailed quiz analytics: average score per quiz, pass/fail ratio, time taken
- Support quiz import from Google Forms or Excel (.xlsx) in addition to CSV/TXT

14. CONCLUSION

The Quiz Management System successfully achieves its primary objective of providing a digital platform for quiz creation, management, and evaluation. The system effectively replaces traditional paper-based quizzes with a streamlined desktop application that benefits both administrators and students.

From a technical perspective, the project demonstrates the practical application of several important software engineering concepts:

- Object-Oriented Programming through well-defined model classes with encapsulation
- Design Patterns including the DAO pattern for clean separation of business and data logic
- Java Swing GUI development with consistent theming and user experience
- JDBC for robust, parameterised database connectivity to MySQL
- Custom exception handling for graceful error management
- Maven-based project structure for organised, maintainable code

The MySQL database with five tables (users, quizzes, questions, attempt, results) provides a solid data foundation. The system has been tested successfully across all major modules including authentication, quiz creation, quiz attempt, and score viewing.

While the current version has certain limitations — such as plain-text password storage, lack of a quiz timer, and the unused results table — these present clear pathways for future improvement. The foundation laid by this project makes it straightforward to migrate to a web-based architecture or to enhance security and feature depth.

Overall, the Quiz Management System is a functional, well-structured application that fulfils the requirements of an academic project while also serving as a practical learning exercise in full-stack Java desktop development.

15. REFERENCES

15.1 Books

- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.
- Horstmann, C. S. (2019). Core Java Volume I – Fundamentals (11th ed.). Pearson.
- Deitel, P. & Deitel, H. (2017). Java: How to Program (10th ed.). Pearson.
- Ramez Elmasri & Shamkant B. Navathe. (2015). Fundamentals of Database Systems (7th ed.). Pearson.

15.2 Online Resources

- Oracle Java Documentation – <https://docs.oracle.com/javase/11/>
- Java Swing Tutorial – <https://docs.oracle.com/javase/tutorial/uiswing/>
- MySQL 8.0 Reference Manual – <https://dev.mysql.com/doc/refman/8.0/en/>
- JDBC Developer's Guide – <https://docs.oracle.com/javase/tutorial/jdbc/>
- Apache Maven Documentation – <https://maven.apache.org/guides/>

15.3 Technologies Referenced

Technology	Version	Official Website
Java Development Kit (JDK)	11+	https://www.oracle.com/java/
Java Swing (GUI Toolkit)	Part of JDK	https://docs.oracle.com/javase/tutorial/uiswing/
MySQL Community Server	8.0+	https://www.mysql.com/
MySQL Connector/J (JDBC Driver)	8.x	https://dev.mysql.com/downloads/connector/j/
Apache Maven	3.6+	https://maven.apache.org/

— END OF REPORT —